Technical University of Denmark

Written examination date: May 23, 2025

Course title: Formal Languages and Semantics

Course number: 02143

Aids allowed: All aid

Exam duration: 2 hours

Weighting: 7-step scale

RULES & RECOMMENDATIONS:

R1 You must hand-in your submission **digitally**.

R2 Submissions must be hand-in as follows:

- **Main Document**: Upload here your PDF. The name of the file must be **main.pdf**.
- **Attachments**: Upload here the **files** folder zipped into a file called **files.zip**. The folder should have the same subfolders and files as you downloaded. The files, of course, should contain your solutions. IMPORTANT: remember to upload the **files** folder, not only the sub-folders.

R3 You are allowed to hand-in paper sheets in addition to your digital submission. If you submit some answers on paper, you must clearly indicate in your digital version (at the respective position for the answer) that the solution is provided on paper and on which page.

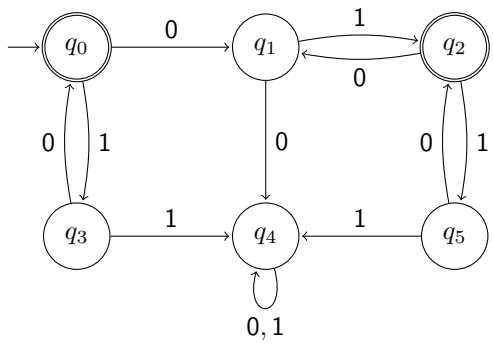
R4 Some exercises require that you provide files that can be used with the tools seen in the course (fm4fun, inspectify, chip, moka). In those cases, your digital submission must follow the precise naming of the files as provided. So, please do not rename the file names. Wrong file names and screenshots are not valid submissions. It is your responsibility to follow carefully the formatting requirements of the files. For example, GCL code with parsing errors will not be accepted.

R5 You must clearly indicate on each page your student number, and which exercise you attempt to solve.

R6 We recommend that you first skim all tasks before deciding in which order you want to solve them.

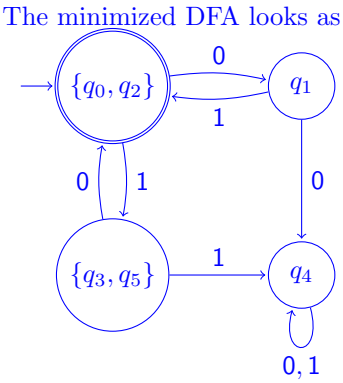
Task 1 – Formal Languages

Alice designs the following DFA A over $\Sigma = \{0, 1\}$



Question 1a: Since Bob has hard time understanding DFAs, Alices decides to try minimizing A as a new equivalent DFA A'. Help Alice obtaining A'. Use the minimisation method seen in the course. You do not need to show the intermediate steps. You just need to provide A' as a transition diagram in the PDF. Each state of A' should be labeled with subsets of states of A.

Solution:
The minimized DFA looks as follows:



The language of this automaton is $\mathcal{L}(\mathsf{A}) = (01 + 10)^*$.

Question 1b: Eve prefers regular expressions over DFA. She designs the following regular expression E: $\mathsf{1}^*(\mathsf{01})^*$.

Alice and Eve meet to discuss how their models relate to each other. Help them by filling the table below. In the empty cell corresponding to language L provide a short-as-possible word that belongs to L. If no such word exists, write `NONE`.

Language L	Shortest word
$\mathcal{L}(\mathsf{A'}) \cap \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{A'}) \cup \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{A'}) \setminus \mathcal{L}(\mathsf{E})$	
$\mathcal{L}(\mathsf{E}) \setminus \mathcal{L}(\mathsf{A'})$	
$\overline{\mathcal{L}(\mathsf{A})} \cup \mathcal{L}(\mathsf{E})$	
$\overline{\mathcal{L}(\mathsf{A})} \cap \overline{\mathcal{L}(\mathsf{E})}$	
$\mathcal{L}(\mathsf{A})^* \cup \mathcal{L}(\mathsf{E})^*$	

Provide the table in text format in file **task1b.txt**, with one line per row. For example, if you think that the answer in all cases is 0101010 write

```
0101010
0101010
0101010
0101010
0101010
0101010
0101010
0101010
```

You can denote the empty string just a blankline.

Solution: The filled table can be found below.

Language	Shortest word
$\mathcal{L}(\mathsf{A'}) \cap \mathcal{L}(\mathsf{E})$	ε
$\mathcal{L}(\mathsf{A'}) \cup \mathcal{L}(\mathsf{E})$	ε
$\mathcal{L}(\mathsf{A'}) \setminus \mathcal{L}(\mathsf{E})$	10
$\mathcal{L}(\mathsf{E}) \setminus \mathcal{L}(\mathsf{A'})$	1
$\overline{\mathcal{L}(\mathsf{A})} \cup \mathcal{L}(\mathsf{E})$	0 / 1
$\overline{\mathcal{L}(\mathsf{A})} \cap \overline{\mathcal{L}(\mathsf{E})}$	10
$\mathcal{L}(\mathsf{A})^* \cup \mathcal{L}(\mathsf{E})^*$	ε

Question 1c: George is a fan of grammars. He has discovered a grammar G in a book on modal logics, which has modalities to reason about events that occurr *once* ($\mathbf{O}$) is and *historically* ($\mathbf{H}$). The grammar G consists of the following productions:

$$
\begin{array}{rcll}
F & \rightarrow & A & (1) \\
 & | & F \wedge F & (2) \\
 & | & A\ R\ N & (3) \\
 & | & \mathbf{O}F & (4) \\
 & | & \mathbf{H}F & (5) \\
\mathbf{HH}F & \rightarrow & \mathbf{H}F & (6) \\
A & \rightarrow & \mathsf{a} & (7) \\
 & | & \mathsf{b} & (8) \\
N & \rightarrow & \mathsf{1} & (9) \\
 & | & \mathsf{2} & (10) \\
R & \rightarrow & \leq & (11) \\
 & | & \geq & (12)
\end{array}
$$

where

- The non-terminal symbols are F, A, N and R;
- The terminal symbols are $\wedge$, $\mathbf{O}$, $\mathbf{H}$, a, b, $\mathsf{1}$, $\mathsf{2}$, $\leq$ and $\geq$;
- The starting symbol is F;
- The numbers of the right indicate the number of the production.

George wants to show the grammar G to Alice and Eve, who are familiar with context-free grammars. But...

... is grammar G context-free? Provide in file **task1c.txt** the smallest set of productions that need to be removed to make G context-free. Write one number per line. For example if you think that the first 3 production should be removed to make the grammar context-free write

```
1
2
3
```

If you think that the grammar is already context-free, leave the file empty.

Solution:

```
6
```

Question 1d: Alice wants to implement a parser for the grammar G' obtained from G after removing the productions identified in (c).

She asks George a relevant question: Does the grammar G have ambiguities? Help Bob answering the question. If your answer is YES, provide in file **task1d.txt** pairs of productions that yield ambiguities. For example, if you think that there is an ambiguity stemming from some combination of the productions 1 and 2, and another one stemming from some combination of 1 and 3 write

```
1,2
1,3
```

If you think there is no ambiguity, leave the file empty.

In addition to the file, please provide a short explanation in the PDF. If you think that there are no ambiguities, please write 1-3 lines explaining why. If you think that there are ambiguities, select one and showcase an example of the ambiguity as we did in the lectures. If you need to write text, limit yourself to 1-3 lines.

Solution:

```
2,2
2,4
2,5
```

As en example consider the ambiguity stemming from productions 2 and 5. Consider string $a \wedge \mathbf{H}b$. As the precedence between these two operations is not defined, we can obtain two separate left-most derivations:

- $F \Rightarrow F \wedge F \Rightarrow a \wedge F \Rightarrow a \wedge \mathbf{H}F \Rightarrow a \wedge \mathbf{H}b$
- $F \Rightarrow F \wedge F \Rightarrow F \wedge \mathbf{H}F \Rightarrow F \wedge \mathbf{H}b \Rightarrow a \wedge \mathbf{H}b$

Similarly we could find two different parse trees (one per each derivation).

Question 1e: We now want to help Bob address question (d) more in depth. If your answer to (d) is NO, expand your explanation in (d) by providing a formal proof. If your answer to (d) is YES, fix the ambiguity you chose to demonstrate in (d) by a new grammar that removes said ambiguity. Explain the changes in 1-3 lines.

Solution:

To fix the specific ambiguity provided in the solution to (d), the updated grammar is obtained by establishing the precedence of operators between the operators in productions 2 and 5 using the stratification method:

$$
\begin{array}{rcl}
F_0 & \rightarrow & F_1 \mid F_0 \wedge F_0 \\
F_1 & \rightarrow & A \mid A \ \odot \ N \rightarrow \mathbf{O}F_1 \mid \mathbf{H}F_1 \\
A & \rightarrow & \mathsf{a} \mid \mathsf{b} \mid \ldots \\
N & \rightarrow & \mathsf{1} \mid \mathsf{2} \mid \ldots \\
\odot & \rightarrow & \leq \mid \geq \mid = \mid \neq \mid > \mid <
\end{array}
$$

Other ambiguities can be dealt with similarly.

Task 2 – Programming Language Semantics

Alice wants to run GCL programs efficiently. For that purpose she wants to build compiler that translates GCL code into a bytecode language. She has been told that bytecode languages allow at most one binary operation in each variable update. Following the recommendation, she prepares the following function bin() that transforms GCL programs into GCL programs:

```
        bin(skip)  =  skip
        bin(v:=u)  =  v:=u
     bin(v:=a1@u)  =  bin(v:=a1) ; v:=v@u
     bin(v:=u@a1)  =  bin(v:=a1) ; v:=u@v
    bin(v:=a1@a2)  =  bin(v:=a1) ; bin(v:=v@a2)
      bin(v:=-a1)  =  bin(v:=-1*a1)
     bin(v:=(a1))  =  bin(v:=a1)
     bin(c1 ; c2)  =  bin(c1) ; bin(c2)
    bin(if gc fi)  =  if bin(gc) fi
    bin(do gc od)  =  do bin(gc) od
     bin(p -> c1)  =  p -> bin(c1)
  bin(gc1 [] gc2)  =  bin(gc1) [] bin(gc2)
```

where

- `v` is any variable,
- `u` is an integer number or a variable,
- `a1`, `a2` are arithmetic expressions,
- `@` is any of the binary arithmetic expressions of GCL (`+,*,/,-,...`),
- `c1`, `c2` are commands,
- `p` is a boolean predicate, and
- `gc`, `gc1`, `gc2` are guarded commands.

Question 2a: Provide in file **task2a.txt** the GCL program that results from applying function bin() to the GCL program below.

```
x := a + b * c - d
```

where `x`, `a`, `b`, `c`, and `d` are variables.

Solution:

```
x:=b;
x:=x*c;
x:=a+x;
x:=x-d
```

Question 2b: Bob suspects that there is something wrong in function bin(). Confirm Bob's suspects by providing the following:

- a memory σ_1 in file **task2b1.txt**;
- a memory σ_2 in file **task2b2.txt**;
- a memory σ_3 in file **task2b3.txt**;
- a deterministic GCL program C in file **task2b4.txt**;
- the transformed program bin(C) in file **task2b5.txt**;

such that

1. C has a complete execution sequence starting with the initial memory σ_1 and ending in memory σ_2;
2. bin(`C`) has a complete execution sequence starting with the initial memory σ_1 and ending in memory σ_3;
3. $\sigma_2 \neq \sigma_3$.

The memories σ_1, σ_2 and σ_3 must be provided as GCL boolean predicates (with no short-circuit operations). For example, the memory $x \mapsto 1, y \mapsto 2, z \mapsto 3$ must be provided as

```
x=1 & y=2 & z=3
```

HINT: Loops are not needed. Aim at the smallest possible example.

In addition to the files, please explain in 1-3 lines in the PDF what is wrong with function bin().

Solution:

The main issue with function bin() is that intermediate values of expressions are stored in variables that may appear in the expression being evaluated. The following minimal example illustrates the issue:

Command C:

```
x := 1 + x
```

Command bin(C):

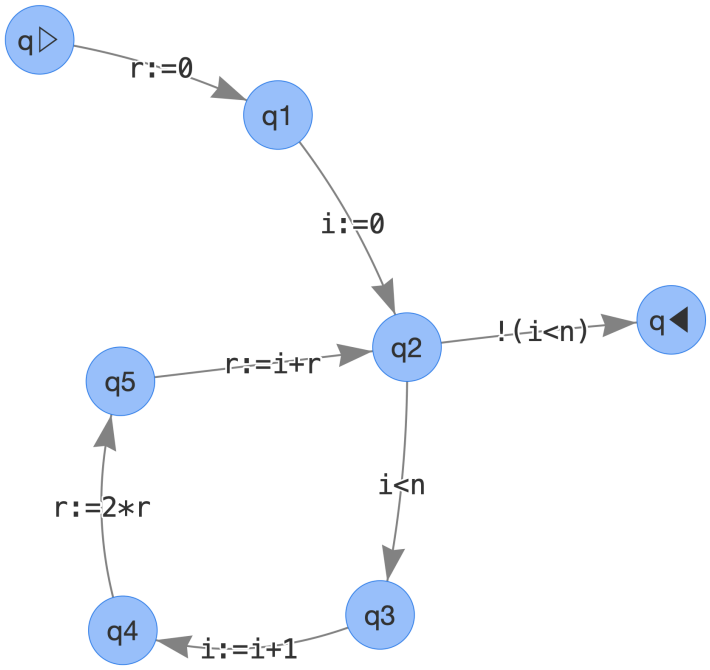
```
x := 1;
x := x + x
```

Not that the above program starts by setting `x` to 1, hence overwriting the value of `x`.

Memories σ_1, σ_2, and σ_3, one per line:

```
x = 0
x = 1
x = 2
```

Question 2c: Alice finds the following Program Graph on Bob's whiteboard



It is the program graph bin(C) of some command C. Your task is to help Alice discover C. Provide C in file **task2c.txt**.

Solution:

```
r:=0;
i:=0;
do i<n -> i:=i+1;
          r:=r+(2*i)
od
```

Question 2d: Alice and Bob engage in a discussion about properties of GCL commands, program graphs and the function bin(). Help them by answering with `True` or `False` the following questions :

1. For all commands C, if C is evolving then bin(C) is evolving.
2. For all commands C, if C constitutes a deterministic system then bin(C) constitutes a deterministic system.
3. For all commands C it holds that bin(C) = bin(bin(C)).
4. There exist commands C and C' such that bin(C) = bin(C').
5. For all commands C, the Program Graph of C is the same as the Program Graph of bin(C).
6. For all commands C, the Program Graph of C has at least the same number of nodes than the Program Graph of bin(C).
7. Given a program graph PG, if there exists a command C whose program graph is precisely PG, then there exists also some command C' such that PG is the Program Graph of bin(C').
8. For all commands C and C', if C, bin(C), and bin(C') have the same complete execution sequences, then $C = C'$.

Provide your answers in file **task2d.txt**, writing one answer (`True`/`False`) per line. For example, if you believe only the even answers are correct, just write

```
False
True
False
True
False
True
False
True
```

In addition to the file, please select one `True` statement and one `False` statement and explain (in 1-3 lines each) in the PDF your answer.

Solution:

1. False: The main reason is that bin() is incorrect. We can easily construct a command C that is evolving while bin(C) is not: `x:=0; x:=1+x; if x=1 -> skip fi`

2. False: For the same reason as above. We can easily construct a command C that is deterministic while $\mathsf{bin}(C)$ is not: `x:=0; x:=1+x; if x=1 -> skip [] x = 2 -> x=0 [] x= 2 -> x =1 fi`

3. False: A simple example is `x := 1 + 1`.

4. True: Simple examples are `x := 0` and `x := (0)`.

5. False: A simple example is `x := 1 + 1`.

6. True: One just needs to notice that $\mathsf{bin}()$ keeps or doubles statements at every line of its definition.

7. False: The PG could have an assignment with more than one binary operation, which would never happen when we apply function $\mathsf{bin}()$.

8. False: C and C' can differ in branches that get stuck. Trivial example: `x:  = 1/0` and `x:=2/0`.

Question 2e: Alice wrote the below theorem and proof as part of her attempt to show the correctness of bin().

Theorem 1. *For all memories σ, all u (either integers or variables in $dom(\sigma)$), and all arithmetic expressions a, a_1, a_2, the following properties hold:*

$$\begin{array}{rcll} \mathcal{A}[\![u]\!]\sigma & = & \mathcal{A}[\![u]\!]\sigma & (P1) \\ \mathcal{A}[\![a@u]\!]\sigma & = & \mathcal{A}[\![x@u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & (P2) \\ \mathcal{A}[\![u@a]\!]\sigma & = & \mathcal{A}[\![u@x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & (P3) \\ \mathcal{A}[\![a_1@a_2]\!]\sigma & = & \mathcal{A}[\![x@a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & (P4) \end{array}$$

Proof by cases:

Case P1 $\mathcal{A}[\![u]\!]\sigma \quad = \quad \mathcal{A}[\![u]\!]\sigma$ (step 1)

Case P2 (case P3 is similar and skipped)

$$\begin{array}{rll} & \mathcal{A}[\![x@u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma]@\mathcal{A}[\![u]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma] & \text{(step 2)} \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a]\!]\sigma]@\mathcal{A}[\![u]\!]\sigma & \text{(step 3)} \\ = & \mathcal{A}[\![a]\!]\sigma@\mathcal{A}[\![u]\!]\sigma & \text{(step 4)} \\ = & \mathcal{A}[\![a@u]\!]\sigma & \text{(step 5)} \end{array}$$

Case P4

$$\begin{array}{rll} & \mathcal{A}[\![x@a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & \\ = & \mathcal{A}[\![x]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma]@\mathcal{A}[\![a_2]\!]\sigma[x \mapsto \mathcal{A}[\![a_1]\!]\sigma] & \text{(step 6)} \\ = & \mathcal{A}[\![a_1]\!]\sigma@\mathcal{A}[\![a_2]\!]\sigma & \text{(step 7)} \\ = & \mathcal{A}[\![a_1@a_2]\!]\sigma & \text{(step 8)} \end{array}$$

Help Alice determining which of the 8 steps in the proof is correct or incorrect. A proof step is correct if the equality stated in the same line holds.

For simplicity, you can assume that all expressions are defined and that Alice has been using a simplified version of the definition of $\mathcal{A}$, e.g.

$$\mathcal{A}[\![a_1 + a_2]\!]\sigma = \mathcal{A}[\![a_1]\!]\sigma + \mathcal{A}[\![a_2]\!]\sigma$$

Provide your answers in file **task2e.txt**, writing one answer (`Correct`/`Incorrect`) per line. For example, if you believe only the even steps are correct, just write

```
Incorrect
Correct
Incorrect
Correct
Incorrect
Correct
Incorrect
Correct
```

In addition to the file, please select one correct step and one incorrect step and explain (in 1-3 lines each) in the PDF your answer.

Solution:

```
Correct
Correct
Incorrect
Correct
Correct
Correct
Incorrect
Correct
```

Incorrect steps wrong apply the definition of $\mathcal{A}$ ignoring that u and x could be the same variable. The rest of the steps are trivial, mostly applying the definition of $\mathcal{A}$ correctly.